

1. Project Overview

In this project, you will design and implement a small management system using **procedural programming in standard C**. You must select **ONE** of the following project options:

- **Option A:** Hospital Management System
- **Option B:** Restaurant Ordering System

The objective is to demonstrate your understanding of fundamental C programming concepts, including:

- Structures (**struct**)
- Variables and data types
- Arrays
- Conditional statements (**if-else**, **switch**)
- Loops (**for**, **while**)
- Functions
- String handling
- Searching and updating records

2. Option A — Hospital Management System

2.1 Required Data Structures

Define appropriate **struct** types for the following entities.

2.1.1 Patient

The **Patient** structure should contain, at minimum:

Field	Description
Patient ID	Unique identifier
Name	Patient's name
Age	Patient's age
Condition	Medical condition
Assigned Doctor ID	ID of the assigned doctor
Room Number	Assigned room
Days Admitted	Number of days admitted
Total Bill	Current/final bill
Status	Admitted or Discharged

2.1.2 Doctor

The **Doctor** structure should contain, at minimum:

Field	Description
Doctor ID	Unique identifier
Name	Doctor's name
Specialization	Medical specialization
Availability	Available or unavailable
Consultation Fee	Fee charged by the doctor

2.2 Required System Actions

Your program must implement the following actions.

2.2.1 1. Register Patient

Add a new patient record to the patient array.

The program should:

- Ask the user for patient information.
- Assign or accept a unique Patient ID.
- Store the new record.
- Set the patient's status to **Admitted**.

2.2.2 2. Search Doctor by Specialization

Ask the user for a specialization and display all doctors with that specialization who are currently available.

For example:

```
1 Enter specialization: Cardiology
2
3 Available Doctors:
4 ID      Name      Fee
5 101     Dr. Rahman  1500
6 104     Dr. Karim   1200
```

2.2.3 3. Assign Doctor to Patient

Ask for a Patient ID and Doctor ID.

The program should verify that:

- The patient exists.
- The doctor exists.
- The doctor is available.

If all conditions are satisfied, assign the doctor to the patient and update the doctor's availability.

2.2.4 4. Search Patient

Allow the user to search for a patient using the **Patient ID**.

Display the patient's complete information if the record exists.

2.2.5 5. Update Patient Information

Allow the user to update selected patient information, such as:

- Name
- Age
- Medical condition
- Room number
- Number of days admitted

The Patient ID should not be changed.

2.2.6 6. Discharge Patient and Calculate Bill

Calculate the patient's final bill using:

$$\text{Total Bill} = (\text{Days Admitted} \times \text{Daily Room Rate}) + \text{Doctor Consultation Fee}$$

The program should:

1. Find the patient.
2. Calculate the final bill.
3. Change the patient's status to **Discharged**.
4. Make the assigned doctor available again.
5. Display a receipt.

Example:

```

1 =====
2           HOSPITAL BILL
3 =====
4 Patient ID       : 1005
5 Patient Name     : Ahmed Hasan
6 Room Number     : 203
7 Days Admitted   : 4
8 Room Charge     : Tk. 8000
9 Doctor Fee      : Tk. 1500
10 -----
11 TOTAL BILL      : Tk. 9500
12 Status         : Discharged
13 =====

```

2.2.7 7. Display Hospital Summary

Display a summary containing:

- Total number of patients
- Number of admitted patients
- Number of discharged patients
- Number of available doctors
- Number of unavailable doctors
- Total revenue generated from discharged patients

3. Option B — Restaurant Ordering System

3.1 Required Data Structures

Define appropriate **struct** types for the following entities.

3.1.1 MenuItem

The **MenuItem** structure should contain, at minimum:

Field	Description
Item ID	Unique identifier
Item Name	Name of food/beverage
Category	Food, Beverage, Dessert, etc.
Price	Price per item
Stock Quantity	Available quantity

3.1.2 Order

The **Order** structure should contain, at minimum:

Field	Description
Order ID	Unique order identifier
Table Number	Customer's table
Ordered Item IDs	IDs of ordered items
Quantities	Quantity of each item
Total Amount	Total order price
Status	Pending, Completed, or Cancelled

3.2 Required System Actions

Your program must implement the following actions.

3.2.1 1. Add New Menu Item

Add a new food or beverage item to the menu inventory.

The program should:

- Accept the item information.
- Store it in the menu array.
- Ensure that the Item ID is unique.

3.2.2 2. Display Menu

Display all available menu items in a formatted table.

```

1 =====
2 ID      Item Name      Category      Price
3 =====
4 101     Chicken Burger  Main          250
5 102     French Fries    Side          120
6 103     Coffee          Beverage      100
7 =====

```

3.2.3 3. Search Menu Item

Allow the user to search for an item using its **Item ID** or **Item Name**.

Display the item's details if it exists.

3.2.4 4. Place Order

Allow a customer to select one or more menu items.

For each selected item, the program should:

1. Search for the item.

2. Ask for the required quantity.
3. Verify sufficient stock.
4. Add the item to the order.
5. Decrease the available stock.
6. Update the order total.

The program should reject an order if the requested quantity exceeds available stock.

3.2.5 5. Apply Seasonal Discount

Apply a specified percentage discount to eligible menu items or an open order.

For example:

$$\text{Discount} = \frac{\text{Discount Rate} \times \text{Total}}{100}$$

$$\text{Final Total} = \text{Total} - \text{Discount}$$

3.2.6 6. Update Menu Item

Allow the restaurant administrator to update:

- Item name
- Category
- Price
- Stock quantity

The Item ID should remain unchanged.

3.2.7 7. Cancel Order

Allow the user to cancel an existing order.

When an order is cancelled:

- Change its status to **Cancelled**.
- Return the ordered quantities to the corresponding menu item's stock.
- Do not count the cancelled order as completed revenue.

3.2.8 8. Generate Order Receipt

Display a formatted receipt containing:

- Order ID
- Table number
- Ordered items
- Quantity
- Unit price
- Subtotal
- Discount
- Final total

- Order status

Example:

```

1 =====
2                RESTAURANT RECEIPT
3 =====
4 Order ID       : 2007
5 Table Number   : 12
6
7 Item           Qty    Price    Total
8 -----
9 Chicken Burger  2      250      500
10 French Fries   1      120      120
11 Coffee         2      100      200
12 -----
13 Subtotal              820
14 Discount (10%)       -82
15 -----
16 TOTAL                738
17 =====
18 Status: Completed
19 =====

```

3.2.9 9. Display Restaurant Summary

Display:

- Total number of menu items
- Total number of orders
- Completed orders
- Cancelled orders
- Total revenue
- Number of items currently out of stock

4. Program Menu

Your program should provide a menu that allows the user to repeatedly perform different operations.

For example:

```

1 =====
2                HOSPITAL MANAGEMENT SYSTEM
3 =====
4
5 1. Register Patient
6 2. Search Patient
7 3. Search Doctor
8 4. Assign Doctor
9 5. Update Patient
10 6. Discharge Patient
11 7. Hospital Summary
12 8. Exit
13
14 Enter your choice:

```

Use a **switch** statement to handle the user's selection.

The program should continue running until the user selects **Exit**.

5. Programming Requirements

5.1 Data Representation

- Use `struct` for the major entities.
- Use arrays to store multiple records.
- Use appropriate C data types for each field.

5.2 Control Structures

Your program should demonstrate appropriate use of:

- `if-else`
- `switch`
- `for`
- `while`

5.3 Functions

Break your program into meaningful functions.

For example:

```
1 void registerPatient();
2 void searchPatient();
3 void updatePatient();
4 void dischargePatient();
5 void displaySummary();
```

Do **not** write the entire program inside `main()`.

5.4 Input Validation

Your program should handle common invalid inputs, such as:

- Invalid menu choices
- Non-existent IDs
- Negative quantities
- Negative prices
- Insufficient stock
- Attempting to perform an operation on an invalid record

6. Submission Guidelines

Submit the following:

1. **C source code** (`.c` file)
2. **Short documentation** explaining:
 - System overview
 - Structures used
 - Major functions
 - Sample input/output